

EXPERIMENT 1B

PERFORMING CRISP-DM AND SEMMA

Aim :

To apply the CRISP-DM (Cross-Industry Standard Process for Data Mining) and SEMMA (Sample, Explore, Modify, Model, Assess) methodologies for systematically understanding, preparing, analyzing, modeling, and evaluating data to extract meaningful insights and support effective decision-making.

Algorithm :

1. Apply the **CRISP-DM** framework by understanding the problem, collecting and preparing the data, building the model, evaluating the results, and presenting the findings.
2. Apply the **SEMMA** framework by sampling the data, exploring its characteristics, modifying and preprocessing it, building the model, and assessing its performance.
3. Compare and interpret the outcomes obtained using both **CRISP-DM** and **SEMMA** methodologies.
4. Draw conclusions based on the analysis and communicate the results for informed decision-making.

Code - CRISP-DM:

1. TO LOAD THE DATASET

```
df = pd.read_csv("KaggleV2-May-2016.csv")
```

```
df.head()
```

	PatientId	AppointmentID	Gender	ScheduledDay	AppointmentDay	Age	Neighbourhood
0	2.987250e+13	5642903	F	2016-04-29T18:38:08Z	2016-04-29T00:00:00Z	62	JARDIM DA PENHA
1	5.589978e+14	5642503	M	2016-04-29T16:08:27Z	2016-04-29T00:00:00Z	56	JARDIM DA PENHA
2	4.262962e+12	5642549	F	2016-04-29T16:19:04Z	2016-04-29T00:00:00Z	62	MATA DA PRAIA
3	8.679512e+11	5642828	F	2016-04-29T17:29:31Z	2016-04-29T00:00:00Z	8	PONTAL DE CAMBURI
4	8.841186e+12	5642494	F	2016-04-29T16:07:23Z	2016-04-29T00:00:00Z	56	JARDIM DA PENHA

2. TO PRINT TAIL AND ANALYSE SHAPE

```
df.tail()
```

	PatientId	AppointmentID	Gender	ScheduledDay
110522	2.572134e+12	5651768	F	2016-05-03T09:15:35Z
110523	3.596266e+12	5650093	F	2016-05-03T07:27:33Z
110524	1.557663e+13	5630692	F	2016-04-27T16:03:52Z
110525	9.213493e+13	5630323	F	2016-04-27T15:09:23Z
110526	3.775115e+14	5629448	F	2016-04-27T13:30:56Z

3. TO PRINT INFO OF THE DATASET

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 110527 entries, 0 to 110526
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   PatientId             110527 non-null float64
1   AppointmentID          110527 non-null int64
2   Gender                 110527 non-null object
3   ScheduledDay           110527 non-null object
4   AppointmentDay         110527 non-null object
5   Age                   110527 non-null int64
6   Neighbourhood          110527 non-null object
7   Scholarship            110527 non-null int64
8   Hipertension           110527 non-null int64
9   Diabetes               110527 non-null int64
10  Alcoholism             110527 non-null int64
11  Handcap                 110527 non-null int64
12  SMS_received           110527 non-null int64
13  No-show                110527 non-null object
dtypes: float64(1), int64(8), object(5)
memory usage: 11.8+ MB
```

4. TO PRINT COLUMN TO LIST

```
print("Columns in Dataset:\n")
print(df.columns.tolist())
```

Columns in Dataset:

```
['PatientId', 'AppointmentID', 'Gender', 'ScheduledDay',
```

5. PRINT SAMPLE

```
df.sample(5, random_state=42)
```

	PatientId	AppointmentID	Gender	Sche
423	1.694229e+13	5502329	F	2016-03-22 14:36
13205	5.634316e+14	5479190	F	2016-03-16 16:28
19842	7.898258e+13	5660473	F	2016-05-04 16:38
20579	7.222383e+12	5623335	M	2016-04-26 13:54
8043	7.722848e+13	5693345	F	2016-05-13 07:07

6. TO DESCRIBE THE DATASET

```
df.describe().T
```

...		count	mean	std	min	max
	PatientId	21514.0	1.514864e+14	2.615590e+14	9.377953e+04	4.177479e+14
	AppointmentID	21514.0	5.655369e+06	6.310348e+04	5.030230e+06	5.631100e+07
	Age	21514.0	3.670475e+01	2.226491e+01	0.000000e+00	1.800000e+02
	Scholarship	21514.0	8.808218e-02	2.834210e-01	0.000000e+00	0.000000e+00
	Hipertension	21514.0	1.845775e-01	3.879634e-01	0.000000e+00	0.000000e+00
	Diabetes	21514.0	6.251743e-02	2.420986e-01	0.000000e+00	0.000000e+00
	Alcoholism	21514.0	4.703914e-02	2.117275e-01	0.000000e+00	0.000000e+00
	Handcap	21514.0	2.035884e-02	1.523126e-01	0.000000e+00	0.000000e+00
	SMS_received	21514.0	3.079390e-01	4.616519e-01	0.000000e+00	0.000000e+00

7. TO ANALYSE THE MISSING VALUES

```
missing = pd.DataFrame({  
    "Missing Values": df.isnull().sum(),  
    "Percentage": (df.isnull().sum() / len(df)) * 100  
})  
  
missing.sort_values("Missing Values", ascending=False)
```

	Missing Values	Percentage
PatientId	0	0.0
AppointmentID	0	0.0
Gender	0	0.0
ScheduledDay	0	0.0
AppointmentDay	0	0.0
Age	0	0.0
Neighbourhood	0	0.0
Scholarship	0	0.0
Hipertension	0	0.0



8. TO PRINT DUPLICATE ROWS

```
duplicates = df.duplicated().sum()  
print(f"Duplicate Rows: {duplicates}")
```

Duplicate Rows: 0

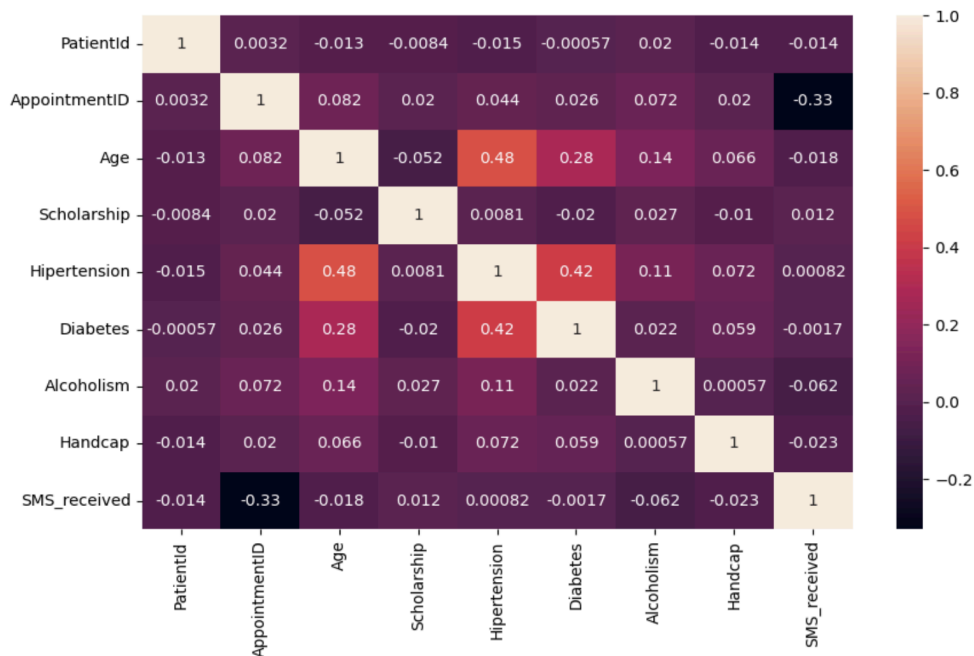
9. TO PRINT UNIQUE VALUES

```
for column in df.columns:
    print(f"{column}: {df[column].nunique()} unique values")
```

PatientId: 15231 unique values
 AppointmentID: 21514 unique values
 Gender: 2 unique values
 ScheduledDay: 20928 unique values
 AppointmentDay: 20 unique values
 Age: 99 unique values
 Neighbourhood: 78 unique values
 Scholarship: 2 unique values
 Hipertension: 2 unique values
 Diabetes: 2 unique values
 Alcoholism: 2 unique values

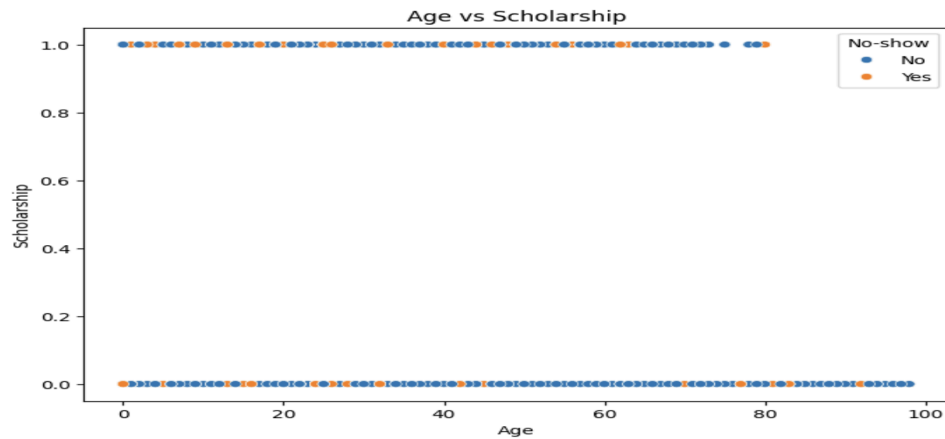
10. TO VISUALISE CORRELATION MATRIX

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```



11. TO ANALYSE REACH VS IMPRESSION

```
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=df,
    x="Age",
    y="Scholarship",
    hue="No-show"
)
plt.title("Age vs Scholarship")
plt.xlabel("Age")
plt.ylabel("Scholarship")
plt.show()
```



12.DATA CLEANING AND PREPROCESSING

```
: df_clean = df.copy()
```

```
:
num_cols = df_clean.select_dtypes(include=['number']).columns
df_clean[num_cols] = df_clean[num_cols].fillna(df_clean[num_cols].median())
cat_cols = df_clean.select_dtypes(include=['object']).columns
df_clean[cat_cols] = df_clean[cat_cols].fillna(df_clean[cat_cols].mode().iloc[0])
```

```
: print("Duplicates before:", df_clean.duplicated().sum())
df_clean.drop_duplicates(inplace=True)
print("Duplicates after:", df_clean.duplicated().sum())
```

Duplicates before: 0

Duplicates after: 0

```
: df_clean['post_datetime'].dtype
```

```
: dtype('O')
```

13.EXTRACT DATA TIME AND CONVERTING THEIR FORMATS

```
df_clean['post_datetime'] = pd.to_datetime(
    df_clean['post_datetime'],
    errors='coerce'
)
df_clean['post_date'] = pd.to_datetime(
    df_clean['post_date'],
    errors='coerce'
)

print(df_clean[['post_datetime', 'post_date']].dtypes)

post_datetime    datetime64[ns]
post_date        datetime64[ns]
dtype: object

df_clean['year'] = df_clean['post_datetime'].dt.year
df_clean['month'] = df_clean['post_datetime'].dt.month
df_clean['day'] = df_clean['post_datetime'].dt.day
df_clean['hour'] = df_clean['post_datetime'].dt.hour
df_clean['day_of_week'] = df_clean['post_datetime'].dt.day_name()

df_clean['post_datetime'].dtype

dtype('<M8[ns]')

df_clean['post_datetime'].head(10)

0    2024-11-30 06:00:00
1    2025-08-15 15:00:00
2    2025-09-11 16:00:00
3    2025-09-18 03:00:00
4    2025-03-21 09:00:00
5    2025-05-06 08:00:00
6    2025-09-21 09:00:00
7    2025-01-06 18:00:00
8    2025-10-13 05:00:00
9    2025-03-10 11:00:00
Name: post_datetime, dtype: datetime64[ns]
```

14.TO

PRINT DATA TIME AFTER CONVERSION

```
df_clean['post_datetime'].head(10)

0    2024-11-30 06:00:00
1    2025-08-15 15:00:00
2    2025-09-11 16:00:00
3    2025-09-18 03:00:00
4    2025-03-21 09:00:00
5    2025-05-06 08:00:00
6    2025-09-21 09:00:00
7    2025-01-06 18:00:00
8    2025-10-13 05:00:00
9    2025-03-10 11:00:00
Name: post_datetime, dtype: datetime64[ns]
```

```
df_clean.drop(columns=['post_id'], inplace=True)
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
categorical_cols = df_clean.select_dtypes(include='object').columns
for col in categorical_cols:
    df_clean[col] = le.fit_transform(df_clean[col])
```

15.FEATURE AND TARGET VARIABLE SEPARATION

Feature and Target Variable Separation

```
X = df.drop("No-show", axis=1)
y = df["No-show"]

print(X.dtypes)
```

PatientId	float64
AppointmentID	int64
Gender	object
ScheduledDay	datetime64[ns, UTC]
AppointmentDay	datetime64[ns, UTC]
Age	float64
Neighbourhood	object
Scholarship	float64
Hipertension	float64
Diabetes	float64
Alcoholism	float64
Handcap	float64
SMS_received	float64
dtype:	object

16. BUILDING THE MODEL AND SPLITTING TRAIN AND TEST



17. BUILDING THE MODEL AND EVALUATING ACCURACY



CODE - SEMMA :

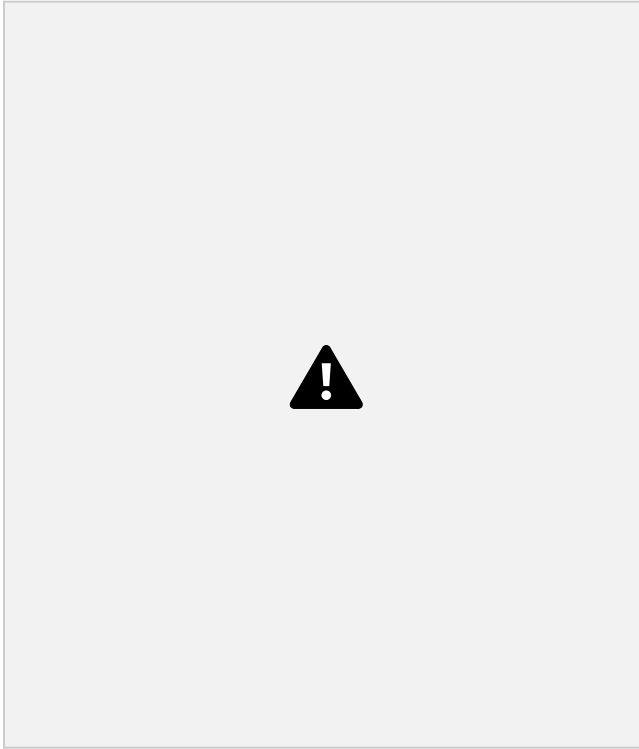
1. TO LOAD THE DATASET AND PRINT HEAD



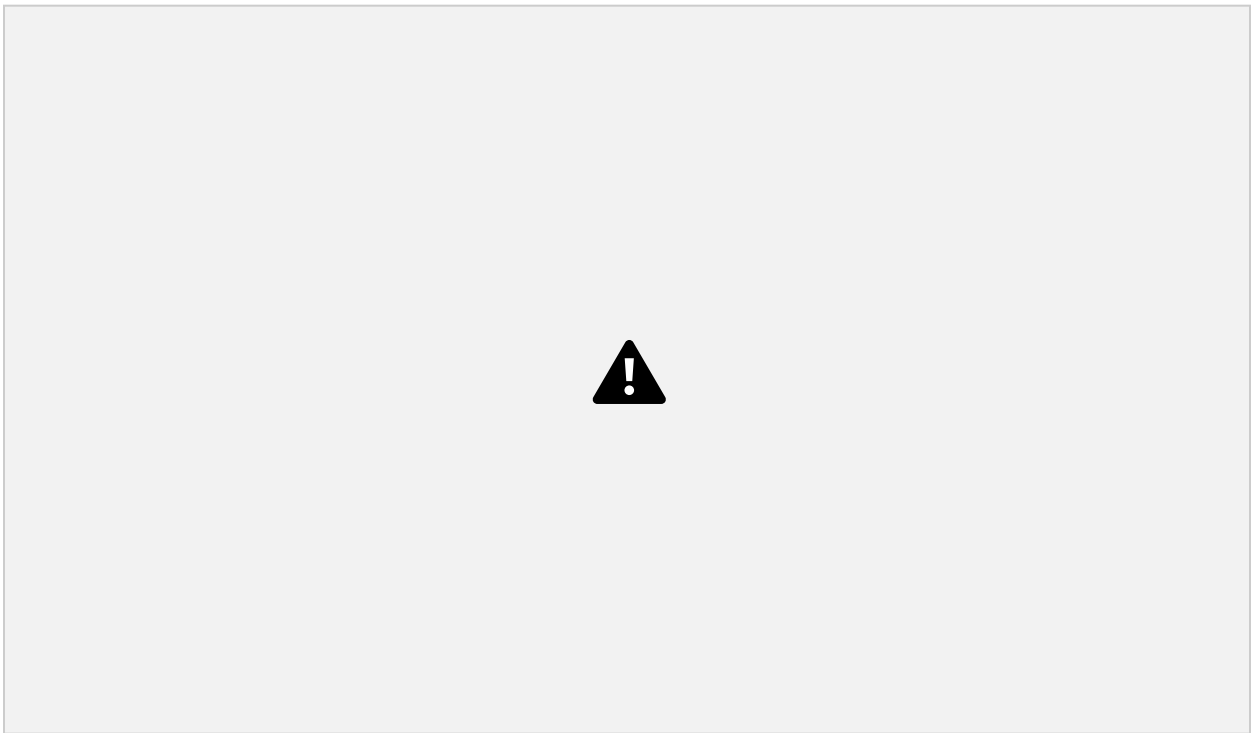
2. TO PRINT TAIL AND EXTRACT SAMPLE



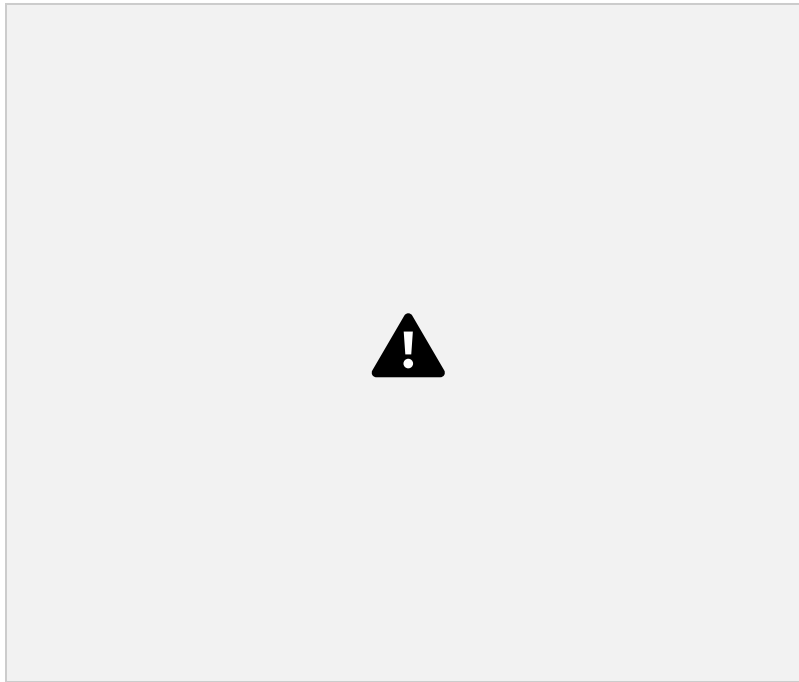
3. TO PRINT INFO OF SAMPLE



4. TO PRINT DESCRIBE

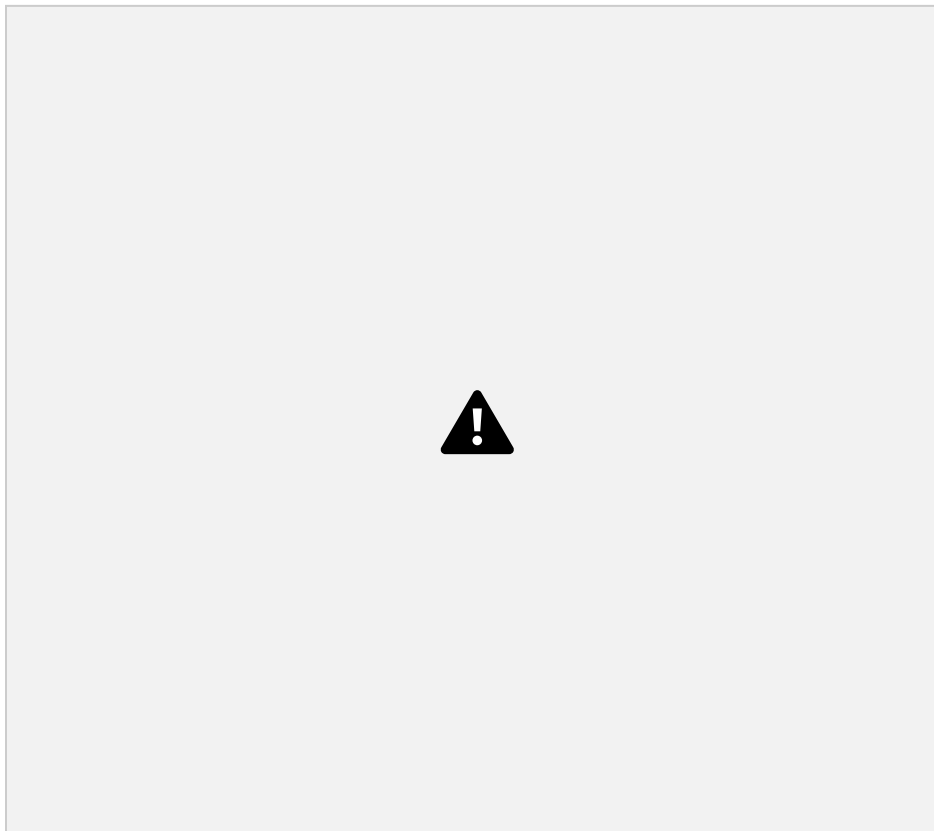


5. TO ANALYSE NULL VALUES AND DUPLICATED ROWS

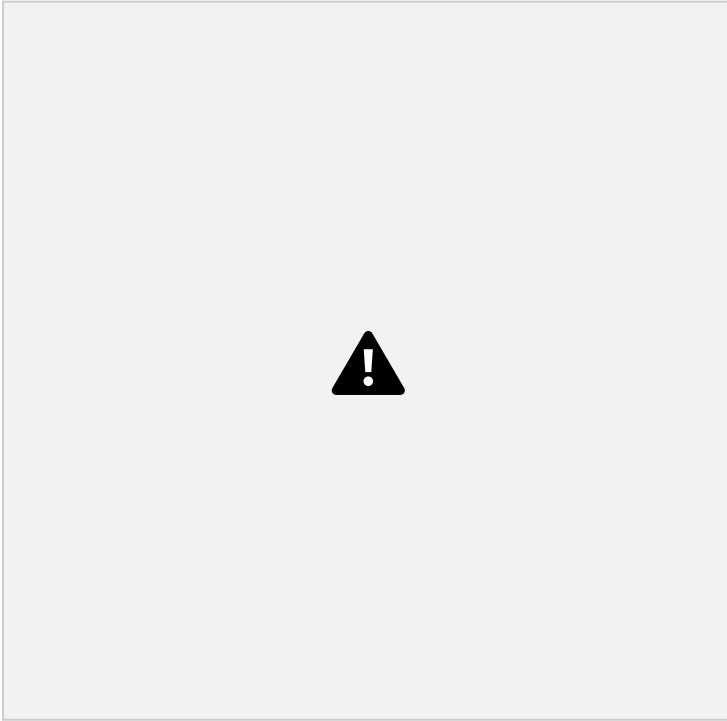


6. TO

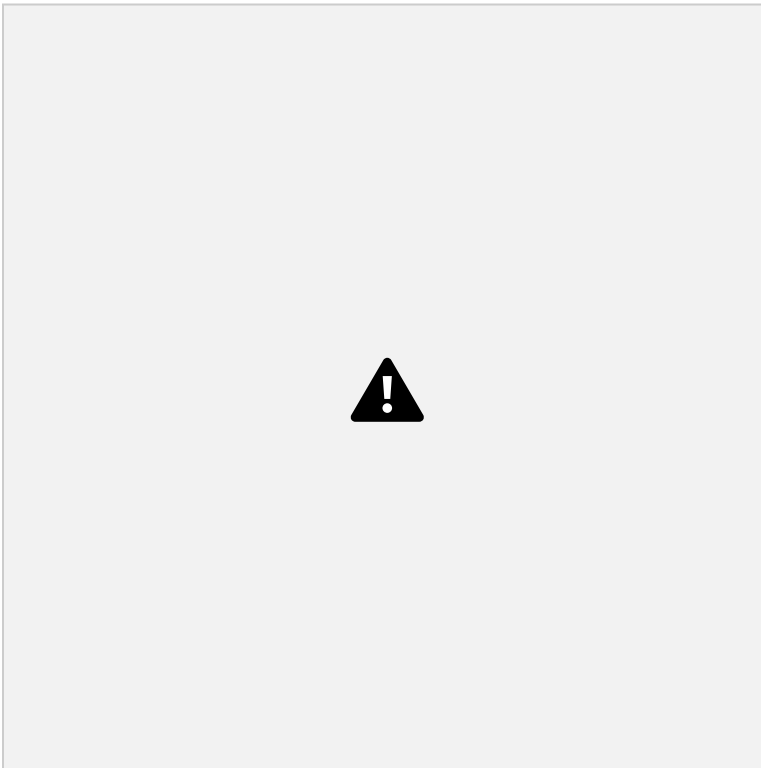
VISUALISE CORRELATION MATRIX



7. Performance Bucket Analysis



8. TO VISUALISE SCATTERPLOT



9. MODIFYING AND CLEANING THE COLUMNS



10.BUILDING THE MODEL



Result :

The program to perform basic mathematical operations is completed successfully